



TILERA[®]

***MULTICORE
DEVELOPMENT
ENVIRONMENT
MiCA
PROGRAMMER'S GUIDE***

DOCUMENT RELEASE 1.00
DOC. No. UG507
MARCH 2012
TILERA CORPORATION

Copyright © 2011-2012 Tiler® Corporation. All rights reserved. Printed in the United States of America.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, except as may be expressly permitted by the applicable copyright statutes or in writing by the Publisher.

The following are registered trademarks of Tiler Corporation: Tiler and the Tiler logo.

The following are trademarks of Tiler Corporation: Embedding Multicore, The Multicore Company, Tile Processor, TILE Architecture, TILE64, TILEPro, TILEPro36, TILEPro64, TILEExpress, TILEExpress-64, TILEExpressPro-64, TILEExpress-20G, TILEExpressPro-20G, TILEExpressPro-22G, iMesh, TileDirect, TILExtreme-Gx, TILEmpower, TILEmpower-Gx, TILEncore, TILEncorePro, TILEncore-Gx, TILE-Gx, TILE-Gx9, TILE-Gx16, TILE-Gx36, TILE-Gx64, TILE-Gx100, TILE-Gx3000, TILE-Gx5000, DDC (Dynamic Distributed Cache), Multicore Development Environment, Gentle Slope Programming, iLib, TMC (Tiler Multicore Components), hardwall, Zero Overhead Linux (ZOL), MiCA (Multicore iMesh Coprocessing Accelerator), and mPIPE (multicore Programmable Intelligent Packet Engine). All other trademarks and/or registered trademarks are the property of their respective owners.

Third-party software: The Tiler IDE makes use of the BeanShell scripting library. Source code for the BeanShell library can be found at the BeanShell website (<http://www.beanshell.org/developer.html>).

This document contains advance information on Tiler products that are in development, sampling or initial production phases. This information and specifications contained herein are subject to change without notice at the discretion of Tiler Corporation.

No license, express or implied by estoppels or otherwise, to any intellectual property is granted by this document. Tiler disclaims any express or implied warranty relating to the sale and/or use of Tiler products, including liability or warranties relating to fitness for a particular purpose, merchantability or infringement of any patent, copyright or other intellectual property right.

Products described in this document are NOT intended for use in medical, life support, or other hazardous uses where malfunction could result in death or bodily injury.

THE INFORMATION CONTAINED IN THIS DOCUMENT IS PROVIDED ON AN "AS IS" BASIS. Tiler assumes no liability for damages arising directly or indirectly from any use of the information contained in this document.

Publishing Information:

Document Number	UG507
Document Release Number	1.00
Date	22 March 2012

Contact Information:

Tiler Corporation
Information info@tilera.com Web Site http://www.tilera.com

Contents

PREFACE	V
 CHAPTER 1 UNDERSTANDING THE MULTICORE IMESH COPROCESSING ACCELERATOR	
1.1 Overview of MiCA	1
1.2 MiCA Accelerator Shims	2
1.3 MiCA Engines	2
1.4 APIs Used with MiCA	2
1.5 MiCA Data Structures and Typedefs	3
 CHAPTER 2 CRYPTOGRAPHIC ACCELERATION	
2.1 Overview of Cryptographic Acceleration	5
2.1.1 Protocols Supported	6
2.1.2 Symmetric-Key Algorithms Available	6
2.1.3 Cryptographic Hash Functions Available	7
2.2 Cryptographic Acceleration Data Structures and Enumerations	7
2.3 Cryptographic Acceleration Functionality	9
2.4 Public Key Authentication (PKA) Acceleration Features	9
2.5 True Random Number Generator Features	10
 CHAPTER 3 COMPRESSION AND DECOMPRESSION	
3.1 Overview of Compression/Decompression	11
3.2 Programming Model	11
3.3 Application Requirements	11
3.4 Creating gzip Compatible Output	12
3.5 For More Information	12
 CHAPTER 4 IPSEC PROCESSING	
4.1 Overview of IPsec (gxcr/ipsec.h)	13
4.2 IPsec Data Structures	13
4.3 IPsec Tokens	14
 CHAPTER 5 EXAMPLE APPLICATIONS	
5.1 DMA Example	15
5.2 Compression and Decompression Example	15
5.3 Cryptographic Example	16

5.4 IPsec Example 16

5.5 Queue Example 17

CHAPTER 6 FREQUENTLY ASKED QUESTIONS (FAQS)

6.1 Basic FAQs 19

GLOSSARY 21

INDEX 23

PREFACE

About This Guide

This guide describes the MiCA™ (Multicore iMesh Coprocessing Accelerator) from Tiler® Corporation from a software perspective.

This guide is a new manual.

This guide applies to MDE Release 4.0.

Intended Audience

This guide is intended for use by Tile Processor™ application developers.

Note: This guide assumes that you are already familiar with the chapter describing the “Common Accelerator Interface (MiCA)” in the *Tile Processor I/O Device Guide for the TILE-Gx Family of Processors* (UG404).

Document Organization

This guide is organized as follows:

- [Chapter 1: Understanding the Multicore iMesh Coprocessing Accelerator](#)
Provides an introduction to MiCA.
- [Chapter 2: Cryptographic Acceleration](#)
Describes how to use the cryptographic acceleration API (declared in `gxcr.h`).
- [Chapter 3: Compression and Decompression](#)
Describes how to use the compression/decompression API.
- [Chapter 4: IPsec Processing](#)
Describes how to use the IPsec processing API (declared in `gxcr/ipsec.h`).
- [Chapter 5: Example Applications](#) describes the MiCA examples in the `examples` directory at `$TILER_ROOT/examples/mica/`.
- [Chapter 6: Frequently Asked Questions \(FAQs\)](#)
Provides FAQs about MiCA and how to use it.

This guide also has a [Glossary](#) and an [Index](#).

MDE Documentation

Note: `TILERA_ROOT` is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is required by many Tiler tools and examples. We recommend that you use the `tile-env` script described in *Getting Started with the Multicore Development Environment* (UG504) to set it correctly.

Tilera MDE Document Index

The Tilera MDE Document Index provides links to online copies of the MDE documentation, in both HTML and PDF formats:

```
$TILERA_ROOT/doc/index.html
```

Location of PDF Files in the Kit

The PDF versions of MDE documents can be found at this location:

```
$TILERA_ROOT/doc/pdf/
```

Man Pages and Info Pages

To view a man page for an x86 tool, run:

```
$ tile-man toolname
```

To view a man page for a Tile tool, run this on a TILE system:

```
$ man toolname
```

To view an info page for a Tile tool, run this on a TILE system:

```
$ info toolname
```

IDE Online Help

The IDE has its own [Tilera IDE Online Documentation](#), which you can access when you use the IDE.

To get to the online help:

- Start `tile-eclipse`.

Select the menu command [Help → Help Contents](#).

- When the online help browser is displayed, select the document [Tilera IDE Online Documentation](#) in the left navigation pane, if it is not already selected.

Technical or Customer Support

You can reach Tilera customer support in either of the following ways:

- Visit the Tilera Web site at <http://www.tilera.com>.
- E-mail questions to support@tilera.com.

Notation Conventions

The following table lists notation conventions used in this guide.

Example	Description
{ }	Alternative required items in syntax descriptions appear within curly brackets. If the contents are separated by a vertical bar, you must choose one of the items.
[]	Optional items in syntax descriptions appear within brackets. If the contents are separated by a vertical bar, you must choose one of the items.
. . .	An ellipse in syntax specifies that the preceding element can be repeated an arbitrary number of times.
\$	This is the system prompt for host-side commands.
#	This is the system prompt for all Tile-side commands and for those x86 host-side commands that must be run as <code>root</code> .
<code>command</code>	A command name appears in the Courier New font
<code>computer output</code>	Computer output appears in the Courier New font.
<i>filename</i>	Non-keyword placeholders appear in italics.
GUI item	A graphical user interface (GUI) item such as a button or menu name appears in red text with the Arial font.
<i>new term</i>	An important word or phrase appears in italics.
Tile Processor™	A processor in the TILE-Gx processor™ family.
TILERA_ROOT	TILERA_ROOT is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is expected by many Tiler tools and examples. We recommend that you use the <code>tile-env</code> script described in <i>Getting Started with the Multicore Development Environment</i> (UG504) to set it correctly.

CHAPTER 1 UNDERSTANDING THE MULTICORE IMESH COPROCESSING ACCELERATOR

This chapter provides an introduction to MiCA™ (Multicore iMesh Coprocessing Accelerator), a set of engines for the Tile Processor:

- [Section 1.1 Overview of MiCA](#)
- [Section 1.2 MiCA Accelerator Shims](#)
- [Section 1.3 MiCA Engines](#)
- [Section 1.4 APIs Used with MiCA](#)
- [Section 1.5 MiCA Data Structures and Typedefs](#)

1.1 Overview of MiCA

MiCA is a set of hardware engines that provide acceleration for these features:

- Encryption/decryption
- Hashing
- Public Key Authentication (PKA) acceleration
- Compression and decompression
- Protocols such as IPsec

The hardware completes these operations several times faster than software would.

The MiCA shim provides a common front end to these various functions.

MiCA services are available to all tiles.

MiCA performs operations on data in memory as specified by an *opcode*. The opcode instructs the hardware to perform an encryption, decryption, compression, or decompression operation.

MiCA can perform multiple operations on a data buffer in one pass; most hardware accelerators cannot.

Tiles communicate with the MiCA engines by means of memory-mapped I/O (MMIO). Software running on the tile configures DTLB (data translation lookaside buffer) entries for MMIO and then accesses the MiCA control registers by means of this interface.

Memory accesses performed by MiCA are translated by an I/O TLB, which is located in the MiCA. This allows completely protected access for operations that user code instructs MiCA to execute.

MiCA supports 40 independent *MiCA hardware contexts*. A hardware context is an entity that contains information (interrupt bindings, state, source and destination data pointers, and other engine-specific information) for processing data. A single tile can use multiple hardware contexts. In practice, however, each tile is typically assigned a single hardware context.

The TILE-Gx36™ processor has two crypto blocks, each with 40 hardware contexts, and two compression/decompression blocks, each with 40 hardware contexts. Thus, the TILE-Gx36 processor has 80 hardware contexts for crypto and 80 hardware contexts for compression/decompression. We generally call these crypto blocks “MiCA shims”.

Each hardware context has a TLB. Each hardware context has 16 I/O TLB entries. You must register any memory that you intend to access by the MiCA shim in these I/O TLBs.

Each MiCA instance contains multiple processing engines and a scheduler to assign requesting hardware contexts to those engines.

In any MiCA operation, data is moved out of and back into L2 caches.

1.2 MiCA Accelerator Shims

There are two types of MiCA accelerator shims:

- Crypto: for symmetric cryptography acceleration functions and Public Key Authentication (PKA)
- Compression: for both compression and decompression operations

The program sets up the MiCA context object to indicate which accelerator to use. The MiCA context object is a software data structure that represents the hardware context.

1.3 MiCA Engines

From a software programming perspective, the MiCA engines are:

For cryptographic MiCA:

- A packet processing engine performing encryption operations
- A packet processing engine performing hashing functions
- A Public Key Authentication (PKA) engine
- A KASUMI engine and a SNOW engine, which share a data channel
- A cryptography-oriented memory-to-memory copy engine
- A cryptography-oriented inverting memory-to-memory copy engine

For compression MiCA:

- Two INFLATE engines
- One DEFLATE engine
- A memory-to-memory copy engine
- An inverting memory-to-memory copy engine

The memory-to-memory and inverting memory-to-memory copy engines are mostly useful for debugging, you could also use them if you really needed an off-core DMA engine.

1.4 APIs Used with MiCA

To employ MiCA features, use these APIs:

- MiCA device API (declared in `gxio/mica.h`): Provides a low-level software programming interface to the MiCA hardware engines. All MiCA operations can be performed using this raw API.

- Crypto Acceleration API (declared in `gxcr.h`): Provides a higher-level software programming interface to the hardware acceleration of bulk encryption, decryption, and hash operations on data.
- IPsec processing API (declared in `gxcr/ipsec.h`): Provides a basic framework for IPsec processing with the MiCA packet processing engine.
- IPsec ESP inbound API (declared in `gxcr/tokens/esp_ip4v_tunnel_ibound.h`): Provides mechanisms to unwrap an IPv4 packet that has been encrypted using the Encapsulating Security Payload (ESP) protocol.
- IPsec ESP outbound API (declared in `gxcr/tokens/esp_ip4v_tunnel_obound.h`): Provides mechanisms to secure an IPv4 packet using the Encapsulating Security Payload (ESP) protocol.

1.5 MiCA Data Structures and Typedefs

These are the data structures and typedefs that all applications use to take advantage of the MiCA features. Individual MiCA APIs can use additional data types:

Data structures

- `gxio_mica_context_t`: A *MiCA context object*, used to manage MiCA hardware resources

Typedef

- `gxio_mica_opcode_t`: A MiCA opcode, used for direct manipulation of a MiCA hardware context

Enumeration

- `gxio_mica_accelerator_type_t`: An enumeration code for the specific type of MiCA shim (cryptographic transformation or compression)

CHAPTER 2 CRYPTOGRAPHIC ACCELERATION

This chapter describes the cryptographic acceleration component of MiCA:

- [Section 2.1 Overview of Cryptographic Acceleration](#)
- [Section 2.2 Cryptographic Acceleration Data Structures and Enumerations](#)
- [Section 2.3 Cryptographic Acceleration Functionality](#)
- [Section 2.4 Public Key Authentication \(PKA\) Acceleration Features](#)
- [Section 2.5 True Random Number Generator Features](#)

You should be familiar with the `gxio_mica` API. For more information, see the *Application Libraries Reference Manual* (UG527).

You should also be familiar with the cryptographic accelerator information in the “Cryptographic Accelerator Interface” chapter in the *Tile Processor I/O Device Guide for the TILE-Gx Family of Processors* (UG404).

2.1 Overview of Cryptographic Acceleration

The cryptographic acceleration API (declared in `gxcr.h`) provides a software programming interface to the hardware acceleration of ciphers and hash functions. The API allows a programmer to create objects that contain the information needed by the MiCA acceleration engine to perform encryption, decryption, and hash operations on data.

The programmer provides a description of the operations to be performed on data, such as which cipher algorithm (if any) to use, which hash function (if any) to use, and whether or not to run the cipher before calculating the hash algorithm.

The data to be processed, the destination for the processed data, and the objects must be in memory that has been registered with the cryptographic shim by means of the `gxio_mica_register_page()` routine.

Use the `gxio_mica_register_page()` routine to register the memory containing the data to be processed, the destination for the processed data, and the software context objects.

The software context objects are of type `gxcr_context_t`, and contain metadata that the MiCA crypto shim needs to process the data, including metadata related to the algorithm and input and result data for the algorithm. Up to 40 hardware context objects are supported. You can have as many `gxcr_context_t` software context objects as you want. A `gxcr_context_t` software context object is not permanently mapped to a hardware context.

Use the routines `gxcr_context_digest0()` and `gxcr_context_digest0_len()` to retrieve the calculated hash value (if you needed one) and the routines `gxcr_context_iv()` and `gxcr_context_iv_len()` to access the key and initialization vector fields for the cipher, if applicable.

The actual packet processing requires the use of a *token* for the engine. The token is a small program: a series of instructions, some of which have fields that vary on a per-SA (Security Association) basis, some of which vary on a per-packet basis. The token is part of the metadata.

The crypto API automatically selects the token for standard operations.

When you use MiCA for crypto, you use `engine_type` 2 for packet processing, `engine_type` 4 for the KASUMI block cipher processing and `engine_type` 5 for SNOW 3G synchronous stream cipher operations.

Note: If you use the `gxcr_` routines, you don't need to know which `engine_type` gets used. If you use the `gxio_` layer directly, you may need to specify these engine types.

2.1.1 Protocols Supported

Table 2-1 shows the protocols supported:

Table 2-1. Protocols Supported

Protocol	Description	Standards
MACsec (802.1AE, the IEEE MAC Security standard)	L2 Ethernet frame encryption/authentication	P802.1AE
IPsec (Internet Protocol Security)	L3 IP packet encryption/authentication	RFC 2403, RFC 2451, RFC 2460, RFC 3566, RFC 3602, RFC 3715, RFC 3686, RFC 3948, RFC 4106, RFC 4301, RFC 4303, RFC 4304, RFC 4308, RFC 4309, RFC 4543, RFC 4835, RFC 4868, RFC 4869
SSL/TLS/DTLS (Secure Sockets Layer/Transport Layer Security/Datagram Transport Layer Security)	L4 encryption/authentication	RFC 3546, RFC 4346, RFC 4347
SRTP (Secure Real-time Transport Protocol)	L3 packet security for RTP	RFC 3711
f8, f9 3GPP (3rd Generation Partnership Project)	Wireless over-the-air security standards (GSM, LTE)	3GPP TS 35.201, 3GPP TS 35.202

2.1.2 Symmetric-Key Algorithms Available

Table 2-2 shows the symmetric-key algorithms available.

Table 2-2. Symmetric-Key Algorithms Available

Algorithm	Key Size	Modes	Standards
AES (Advanced Encryption Standard)	128 bits, 192 bits, 256 bits	CBC, GCM, CTR, ECB, CCM	NIST FIPS 197, RFC 2451, RFC 3566, RFC 3602, RFC 3686, RFC 4308, RFC 4309, RFC 4835, RFC 4869
DES (Data Encryption Standard)	56 bits	CBC	NIST FIPS 86-3, RFC 1829, RFC 2405, RFC 2451, RFC 4309

Table 2-2. Symmetric-Key Algorithms Available (continued)

Algorithm	Key Size	Modes	Standards
3DES (Triple DES)	168 bits	CBC, ECB	NIST FIPS 86-3, RFC 4308, RFC 4835
ARC4 (Alleged RC4) [not yet supported]	40 bits to 256 bits	Stream cipher	IETF draft
KASUMI	128 bits	Stream cipher	3GPP TS 35.202
SNOW 3G	128 bits	Stream cipher	3GPP TS 35.216

2.1.3 Cryptographic Hash Functions Available

[Table 2-3](#) shows the hash functions available.

Table 2-3. Cryptographic Hash Functions Available

Algorithm	Modes	Standards
MD5 (Message Digest algorithm)	Straight hash, HMAC (Hash-based Message Authentication Code)	RFC 1321, RFC 1828, RFC 2401, RFC 2403, RFC 4835
SHA-1 (Secure Hash Algorithm)	Straight hash, HMAC	NIST FIPS 180-2, RFC 2404, RFC 4835
SHA-256 (SHA-2)	Straight hash, HMAC	NIST FIPS 180-2, RFC 2404, RFC 4835
SHA-512 (SHA-2)	Straight hash, HMAC	NIST FIPS 180-2, RFC 2404
AES GMAC	Straight hash	RFC 4543, RFC 4869

For a comprehensive list of RFCs and standards supported in MiCA, see "" in the *Tile Processor I/O Device Guide for the TILE-Gx Family of Processors* (UG404).

2.2 Cryptographic Acceleration Data Structures and Enumerations

These are the data structures and enumerations that applications use with the cryptographic acceleration API (declared in `gxcr.h`):

Data structures

- `gxcr_result_token_t` (the result from the packet processing engine)
- `gxcr_context_t` (a context object that encapsulates data to be sent to the cryptographic acceleration hardware, including metadata related to the algorithm, and input and result data for the algorithm)

Enumeration

- `gxcr_cipher_t` (enumeration code for the available encryption/decryption algorithms). See [Table 2-4](#).
- `gxcr_digest_t` (enumeration code for the available hash algorithms). See [Table 2-5](#).

Tilera makes no recommendations on which algorithms you should use, but by and large the newer, more compute-intensive algorithms (such as AES and SHA1) are more secure.

Table 2-4. Enumeration Values for `gxcr_cipher_t`

Enumeration Value	Description	Standard
<code>GXCR_CIPHER_AES_CBC_number</code> where <i>number</i> is: 128, 192, 256	AES cipher block chaining	NIST FIPS 197, RFC 4308, RFC 4835
<code>GXCR_CIPHER_AES_ECB_number</code> where <i>number</i> is: 128, 192, 256	AES electronic code book	NIST FIPS 197, RFC 3602, RFC 4835
<code>GXCR_CIPHER_AES_ICM_number</code> where <i>number</i> is: 128, 192, 256	AES integer counter mode	NIST FIPS 197, RFC 3711, RFC 4835
<code>GXCR_CIPHER_AES_CCM</code>	AES counter with CBC (using 128 bits)	NIST FIPS 197, RFC 3610, RFC 4309, RFC 4835
<code>GXCR_CIPHER_AES_GCM</code>	AES Galois counter mode (using 128 bits)	NIST SP 800-38D, RFC 4106, RFC 4543, RFC 4835
<code>GXCR_CIPHER_DES_CBC</code>	DES cipher block chaining (using 56 bits)	NIST FIPS 46-3, RFC 2451, RFC 4309, RFC 4835
<code>GXCR_CIPHER_3DES_ECB</code>	3DES electronic code book (using 168 bits)	NIST FIPS 46-3
<code>GXCR_CIPHER_3DES_CBC</code>	3DES cipher block chaining (using 168 bits)	NIST FIPS 46-3, RFC 2451, RFC 4308, RFC 4835
<code>SNOW-3G</code>	Used in 3GPP wireless	3GPP TS 35.216
<code>KASUMI</code>	UMTS, GSM, GPRS wireless	3GPP TS 35.202

Table 2-5. Enumeration Values for `gxcr_digest_t`

Enumeration Value	Description	Standard
<code>GXCR_CIPHER_AES_CCM</code>	AES counter with CBC	RFC 4309
<code>GXCR_DIGEST_MD5</code>	Message Digest algorithm	RFC 1321, RFC 1828, RFC 2401, RFC 2402, RFC 2403, RFC 4835
<code>GXCR_DIGEST_SHA1</code>	Secure Hash Algorithm	NIST FIPS 180-2, RFC 3174

Table 2-5. Enumeration Values for `gxcr_digest_t` (continued)

Enumeration Value	Description	Standard
<code>GXCR_DIGEST_SHA_224</code>	SHA-2 224-bit digest	NIST FIPS 180-2
<code>GXCR_DIGEST_SHA_256</code>	SHA-2 256-bit digest	NIST FIPS 180-2, RFC 4868
<code>GXCR_DIGEST_SHA_384</code>	SHA-2 384-bit digest	NIST FIPS 180-2, RFC 4868
<code>GXCR_DIGEST_SHA_512</code>	SHA-2 512-bit digest	NIST FIPS 180-2, RFC 4868

2.3 Cryptographic Acceleration Functionality

You can access the cipher and hash functionality individually and simultaneously. For example: encrypt-then-hash, hash-then-encrypt (with offsets).

There are two use models:

- Simple algorithm offload: TILE performs header/trailer processing.
- Complex “packet processing” with hardware protocol assist: MiCA handles header/trailer work.

The API in `gxcr.h` is for the bulk processing of data.

Policy lookup and security context management is performed on TILE. For example:

- SPD (Security Policy Database) and SAD (Security Association Database) processing in IPsec
- Handshaking and installing/removing new flows

2.4 Public Key Authentication (PKA) Acceleration Features

The Public Key Authentication (PKA) acceleration component of MiCA is ring-based in hardware. It has these features:

- Ability to perform vector math operations for signature generation and verification
- Support for up to 4096-bit modulus
- Support for RSA (Rivest Shamir Adleman) encrypt/decrypt CRT (Chinese Remainder Theorem), as defined in RSA PKCS #1
- Support for DSA, ECDSA (Elliptic Curve Digital Signature Algorithm), as defined in NIST FIPS 186-3
- Support for DH (Diffie-Hellman) key agreement, as defined in RFC 2631
- Support for ECDH (Elliptic Curve Diffie-Hellman), as defined in NIST SP 800-56A
- Support for ECC (Elliptic Curve Cryptography) encryption/decryption, as defined in RFC 4492

PKA acceleration does not use the MiCA context interface. Instead, you access the hardware queues through Linux.

Library routines are provided to interact with the Linux driver to initialize the queues. The programmer can decide:

- How many queues to use (1, 2, or 4)
- Whether to prioritize queue 0
- The key size for each queue

The application communicates with the Linux driver, which does MMIO writes to I/O device memory to perform operations.

Signature generation and verification is accessed by means of the Linux driver. The driver creates a device file that is opened, written to, and read from an application program.

2.5 True Random Number Generator Features

A true random number generator (TRNG) is used for MiCA cryptographic operations.

The TRNG is accessible by means of the standard Linux interface, `/dev/hw_random`, which is similar to the standard `/dev/random` TRNG. Although the hardware is accessed by means of MMIO, it is protected from user access.

The TRNG is also fed directly into the public key engine for use in PKA acceleration operations. This TRNG derives its entropy from on-chip free-running oscillators.

There is a separate PRNG source for packet processing (initialization vectors, counters, and so forth).

The TRNG does not use the MiCA context interface. Instead, it is accessed through Linux.

CHAPTER 3 COMPRESSION AND DECOMPRESSION

This chapter provides a brief description of the compression/decompression component of MiCA:

- [Section 3.1 Overview of Compression/Decompression](#)
- [Section 3.2 Programming Model](#)
- [Section 3.3 Application Requirements](#)
- [Section 3.4 Creating gzip Compatible Output](#)
- [Section 3.5 For More Information](#)

3.1 Overview of Compression/Decompression

The compression/decompression component of MiCA uses the MiCA device API (declared in `gxio/mica.h`). There is no high-level API for compression/decompression.

The TILE-Gx™ architecture supports hardware acceleration for loss-less compression and decompression operations.

The Raw DEFLATE format (RFC1951) and GZIP file format (RFC1952) are supported by the hardware accelerators for both compression and decompression.

The TILE-Gx36™ implementation supports full-duplex, 10 Gbs compression and 10 Gbs decompression. The TILE-Gx36 implementation provides two TILE-Gx compression controllers. Each controller has one compression engine and two decompression engines. Each controller supports forty hardware context objects.

When you use MiCA for compression, you use `engine_type 2` for compression and `engine_type 3` for decompression.

Variable compression levels are supported.

Static or dynamic Huffman tables are supported.

3.2 Programming Model

Compression is available to all worker tiles.

The worker tile calls the `compress/decompress` function.

There is a low-latency function return.

Data is moved out of and back into L2 caches.

3.3 Application Requirements

The application is responsible for specifying the opcode, transfer size, source data descriptor, and destination data descriptor.

For a compression operation, your application should do the following:

1. Use the `gxio_mica_init()` routine to initialize a MiCA compression context.
2. Use the `gxio_mica_register_page()` routine to register the memory with MiCA so that it can perform DMA operations to and from this memory.
3. Specify the MiCA `opcode.engine_type` of 2, and set the compression parameters (such as `MAX_CHAIN_LENGTH`) within the opcode, or `engine_type` of 3 for decompression.
4. Use the `gxio_mica_start_op()` routine to start the compression.

3.4 Creating gzip Compatible Output

Hardware compressed data in general will not be same as the software compressed data. The contents of a compressed file generated by our hardware will never match the output of the `gzip` utility. Compressing the same file twice using the hardware could produce two different compressed files.

The actual compressed data our hardware emits is in DEFLATE format. To have the COMP hardware create an output file containing header and footer data which is compatible with the `gzip` utility, set Bit 1 in the opcode's `dm_specific` field. You can write the resulting data to a file which the `gzip` utility can decompress it.

To create a compressed file with `gzip` header and footers, set `opcode.dm_specific=0x3`. For example, modify the example compression application (`$TILER_ROOT/examples/mica/comp/app.c`) in the compression section as follows:

```
gxio_mica_opcode_t opcode = {{ 0 }};
opcode.engine_type = 2;           // compression
opcode.size = sizeof(sample_data);
opcode.dm_specific=0x3;
memcpy(src_data, sample_data, sizeof(sample_data));

// Start the compression
gxio_mica_start_op(&cb, src_data, dst_data, NULL, opcode);
```

3.5 For More Information

See the following:

- [Section 5.2 “Compression and Decompression Example” on page 15](#)
- “MiCA Device API (`gxio/mica.h`)” section of the *Application Libraries Reference Manual* (UG527)
- “Common Accelerator Interface (MiCA)” chapter of the *Tile Processor I/O Device Guide for the TILE-Gx Family of Processors* (UG404)
- “Compression Accelerator” chapter of the *Tile Processor I/O Device Guide for the TILE-Gx Family of Processors* (UG404)

CHAPTER 4 IPSEC PROCESSING

This chapter describes the IPsec component of MiCA:

- [Section 4.1 Overview of IPsec \(gxcr/ipsec.h\)](#)
- [Section 4.2 IPsec Data Structures](#)

You should be familiar with the `gxio_mica` API. For more information, see the *Application Libraries Reference Manual* (UG527).

You should also be familiar with the cryptographic accelerator information in the “Cryptographic Accelerator Interface” chapter in the *Tile Processor I/O Device Guide for the TILE-Gx Family of Processors* (UG404).

4.1 Overview of IPsec (gxcr/ipsec.h)

Internet Protocol Security (IPsec) is a protocol suite for securing Internet Protocol (IP) communications by authenticating and encrypting each IP packet of a communication session. IPsec also includes protocols for establishing mutual authentication between agents at the beginning of the session and negotiation of cryptographic keys to be used during the session.

The `gxcr_ipsec` API, declared in `gxcr/ipsec.h`, provides a basic framework for IPsec processing with the MiCA packet processing engine. The MiCA packet processing engine is one of the engines in the cryptographic accelerator.

The `gxcr_ipsec` API allows a programmer to set up a Security Association (SA) data structure based on the parameters in the data structure `gxcr_ipsec_params_t`.

4.2 IPsec Data Structures

These are the data structures that applications use with the IPsec API (`gxcr/ipsec.h`) and the two additional APIs `gxcr/tokens/esp_ipv4_tunnel_ibound.h` and `gxcr/tokens/esp_ipv4_tunnel_obound.h`:

Data structures

- `gxcr_ipsec_params_t` (encapsulates parameters used to set up an IPsec SA)
- `gxcr_ipsec_sa_t` (encapsulates a pointer to the memory that is to be used with the shim for the metadata, and the information about where the token and the context record fields are within that memory)
- `ipsec_esp_ipv4_tunnel_ibound_sa_t` (encapsulates general IPsec SA and operation-specific parameters)
- `ipsec_esp_ipv4_tunnel_obound_sa_t` (encapsulates general IPsec SA and operation-specific parameters)

4.3 IPsec Tokens

The IPsec tokens that are provided by Tiler are specific to ESP IPv4 tunneling with Explicit Initialization Vector (IV). If another type of packet processing is needed, you will need to implement a custom token and its support functions.

To create a token, you will need to understand the in-line packet engine described in the *Tile Processor I/O Device Guide for the TILE-Gx Family of Processors* (UG404). See in particular the “Inline Packet Engine” appendix and the “Inline Packet Engine — TokenExamples” appendix.

Note: Implementing a custom token is a low-level, painstaking operation. You need to study every token field described in the “Inline Packet Engine” appendix and what its function is and write down the exact input you need in every field. There are no debugging tools for custom tokens: you need to experiment and see if your token works as you expect it to.

CHAPTER 5 EXAMPLE APPLICATIONS

This chapter describes the MiCA examples in the Examples directory at `$TILERA_ROOT/examples/mica/`:

- [Section 5.1 DMA Example](#)
- [Section 5.2 Compression and Decompression Example](#)
- [Section 5.3 Cryptographic Example](#)
- [Section 5.4 IPsec Example](#)
- [Section 5.5 Queue Example](#)

5.1 DMA Example

This example demonstrates the use of DMA (direct memory access) using the MiCA shim. It is a good introduction to the use of the low-level `gxio_mica_*` interface.

The example is located here:

- `$TILERA_ROOT/examples/mica/dma/`
- “`mica/dma/app.c`” in the “Example Documentation” chapter of the *Application Libraries Reference Manual* (UG527)

This example shows how to do the following:

- Initialize a MiCA context object (`gxio_mica_context`) for cryptographic acceleration using `gxio_mica_init()`.
- Register the page with MiCA using `gxio_mica_register_page()` so that DMA operations can be performed to and from this memory.
- Start the DMA transfer using `gxio_mica_memcpy_start()`.
- Wait for the operation to complete using `gxio_mica_is_busy()`.
- Compare source and destination data.

5.2 Compression and Decompression Example

This example demonstrates the use of compression and decompression using the MiCA Compression engine and the low-level GXIO API.

Note: The Compression engine shim is currently not supported by the simulator.

The example is located here:

- `$TILERA_ROOT/examples/mica/comp/`
- “`mica/comp/app.c`” in the “Example Documentation” chapter of the *Application Libraries Reference Manual* (UG527)

This application performs the following tasks using routines from the `gxio/mica` API:

1. Initializes a MiCA context object (`gxio_mica_context`) for compression/decompression acceleration using `gxio_mica_init()`.
2. Registers a memory page with MiCA so that it can perform DMA operations to and from this memory using `gxio_mica_register_page()`.
3. Starts compression of the source data using `gxio_mica_start_op()`.
4. Waits for completion of the compression operation using `gxio_mica_is_busy()`.
5. Reads the status of the completed operation using `__gxio_mmio_read()`.
6. Starts decompression of the destination data using `gxio_mica_start_op()`.
7. Waits for completion of the decompression operation using `gxio_mica_is_busy()`.
8. Compares the decompressed data to the original data before compression using `memcmp()`.

5.3 Cryptographic Example

This example shows how to encrypt and decrypt with the AES-CBC-128 symmetric-key algorithm and calculate an MD5 hash algorithm.

In particular, the example shows how to use the cryptographic acceleration functions of the MiCA shim, using the `gxcr` library. It shows how to register memory with the shim, how to create `gxcr_context` data structures for cryptographic and hash functions, how to prepare keys and other data, how to perform the operation, and how to access the results.

The example is located here:

- `$TILERA_ROOT/examples/mica/crypto/`
- `mica/crypto/app.c` in the “Example Documentation” chapter of the *Application Libraries Reference Manual* (UG527)

This application performs the following tasks using routines from the `gxcr/crypto` and `gxio/mica` APIs:

1. Initializes a MiCA context object (`gxio_mica_context`) for cryptographic acceleration using `gxio_mica_init()`.
2. Registers a memory page with MiCA so that it can perform DMA operations to and from this memory using `gxio_mica_register_page()`.
3. Creates a `gxcr_context` object for encrypting data and running a hash algorithm on it using `gxcr_init_context()`.
4. Creates an object for running a hash algorithm on encrypted data and then decrypting the data using `gxcr_init_context()`.
5. Encrypts the source data using `gxcr_do_op()`.
6. Looks at the hash result using `gxcr_context_digest0()`.
7. Decrypts the encrypted data and compares it to the original source using `gxcr_do_op()`.
8. Compares the hash results using `gxcr_context_digest0()`.

5.4 IPsec Example

This example demonstrates how to use the general IPsec acceleration functions and specifically the ESP/IPv4/tunneling inbound and output packet processing on the MiCA shim.

Among other things, it shows how to register memory with the shim.

The example is located here:

- `$TILERA_ROOT/examples/mica/ipsec/`
- “`mica/ipsec/app.c`” in the “Example Documentation” chapter of the *Application Libraries Reference Manual* (UG527)

This application includes the libraries `gxio/mica.h`, `gxcr.h`, and `gxcr/ipsec.h`.

It includes these Tiler-provided tokens:

- `gxcr/tokens/esp_ipv4_tunnel_ibound.h`
- `gxcr/tokens/esp_ipv4_tunnel_obound.h`

This application performs the following tasks using routines from the `gxcr/crypto` and `gxio/mica` APIs:

1. It takes an IPv4 packet and runs it through outbound processing.
2. Outbound processing performs outbound IPsec processing (including assigning a sequence number and adding the SPI to the packet), and encrypts and encapsulates the packet. It appends a digest to the end of the packet.
3. The inbound token validates the SPI, sequence number and digest, and decapsulates and decrypts the packet.
4. It allocates a huge page to hold the metadata and registers the page with MiCA using `gxio_mica_register_page()`.

5.5 Queue Example

This example shows how to use a queuing interface to the IPsec acceleration functions on the MiCA shim.

The example can be modified to work with any MiCA operations.

The `app.c` programs shows how the queue can be used with the IPI (inter-processor interrupt) mechanism (if the preprocessor macro `INTERRUPT_MODEL` is defined to be nonzero) or with the polling mechanism (if it is not).

The example is located here:

- `$TILERA_ROOT/examples/mica/queue/`
- `mica/queue/app.c` in the “Example Documentation” chapter of the *Application Libraries Reference Manual* (UG527)

CHAPTER 6 FREQUENTLY ASKED QUESTIONS (FAQS)

This chapter provides answers to frequently asked questions concerning MiCA.

- [Section 6.1 Basic FAQs](#)

6.1 Basic FAQs

How many pages (per context) can I register using the `gxio_mica_register_page()` routine?

You can register sixteen pages per hardware context, as defined in the `gxio_mica_context_t` structure.

How much memory should be set for metadata of different crypto operations?

The `gxcr_calc_context_bytes()` routine calculates this for you.

How do I free a hardware context and MiCA address space?

They are freed automatically when the application exits.

How do I create custom tokens?

See [Section 4.3 “IPsec Tokens” on page 14](#).

How can I make sure my compression worked?

Use `MICA_COMP_CTX_USER_CONTEXT_STATUS` to check the status.

There are also memory-to-memory copy engines, and inverting memory-to-memory copy engines. Those are mostly useful for debugging but probably could be used if someone really needed an off-core DMA engine.

G GLOSSARY

Term	Definition
MiCA context object	A data structure used to manage MiCA hardware resources.
MiCA hardware context	A hardware entity that contains information (interrupt bindings, state, source and destination data pointers, and other engine-specific information) for processing data.
MMIO	Memory-mapped I/O. MMIO uses the same address bus to access both memory and I/O devices
opcode	MiCA operations code. The opcode instructs the hardware to perform an encryption, decryption, compression, or decompression operation.
Security Association (SA)	The establishment of shared security attributes between two network entities to support secure communication. That is, a logical group of security parameters that enable the sharing of information with another entity.
token	A program run by the packet processing engine.

INDEX

A

audience of this guide [v](#)

C

conventions

notation [vi](#)

customer support [vi](#)

D

document organization [v](#)

I

IDE online help [vi](#)

info pages [vi](#)

M

man pages [vi](#)

MDE Document Index [vi](#)

location of [vi](#)

MDE documentation

location of [vi](#)

N

notation conventions [vi](#)

O

organization of this guide [v](#)

S

support

technical or customer [vi](#)

T

technical support [vi](#)

